

ANÁLISIS DEL USO DE VIBE CODING PARA LA PROGRAMACIÓN CONCURRENTES

Integrantes:

*PALMIERI Marco, QUIROGA PIEGARI Lucila, GONZALEZ Daniela, PARODI Francisco, ALONSO Emiliano,
PASTORI Ximena*

{mpalmieri385, lquirogapiegari, danielapgonzalez, frparodi, calonso, xpastori128}@alumno.unlam.edu.ar

Asignatura:

(03658) PROGRAMACION CONCURRENTES

Fecha - Versión:

27/05/2026 – Versión 1.01

Resumen:

Con los avances de la inteligencia artificial generativa en la generación de código, el concepto de *VibeCoding* ha tomado relevancia en la industria. Esta técnica consiste en desarrollar un programa por medio del dictado de instrucciones en lenguaje natural a una IA, delegándole la implementación técnica. El VibeCoding se ha ido perfeccionando a partir del uso de técnicas de prompting y manejo de contexto, que otorgan un mejor resultado. Por ello, en este informe, aplicaremos una de estas técnicas llamadas *prompt técnico* para evaluar las capacidades de la IA en el desarrollo de aplicaciones concurrentes. Para evaluar los resultados, se aplicarán métricas de software que cuantifiquen el valor aportado por la IA. Si bien en la actualidad han surgido modelos de LLM capaces de generar código de gran valor, su costo asociado es muy elevado. Es por esto que, además, se realizará una comparativa entre el resultado generado por un modelo más inteligente, y costoso, y por otro de menor capacidad, pero menor costo, para comparar la eficiencia de dos modelos a partir de un buen prompt estructurado.

Palabras Clave:

VibeCoding, LLM, Prompt Técnico, Concurrencia, IA

I. CONTEXTO

Este informe corresponde a un trabajo de investigación de la cátedra de Programación Concurrente de Ingeniería Informática, que a su vez corresponde a un estudio mayor sobre la aplicación de Inteligencia Artificial en el desarrollo de software, realizado por dicha cátedra, en la Universidad Nacional de La Matanza.

II. INTRODUCCIÓN

El objetivo del siguiente trabajo es analizar la capacidad de los modelos de inteligencia artificial generativa actuales para la generación de aplicaciones concurrentes. De este modo, se desarrollará una aplicación clasificadora de publicaciones de usuarios, que analizará el sentimiento de la publicación, la clasificará de acuerdo a una categoría de contenido prohibido, y detectará posible spam en el posteo, para finalmente, guardar los resultados en un archivo de salida. Esto se realiza mediante un pipeline para cada etapa del análisis, que contará con un pool de threads que trabajarán de forma concurrente para la optimización de la ejecución.

Esta aplicación será realizada mediante el VibeCoding, a partir del uso de un prompt estructurado según el patrón de *prompt técnico*, que disparará la ejecución de los modelos de inteligencia artificial de la empresa Anthropic, Claude Opus 4.7 y Claude Haiku 4.5, generando así dos resultados a partir del mismo input inicial, que nos servirá para el análisis comparativo de estos dos modelos, mediante el uso de métricas de software para la medición del consumo de CPU y memoria, rendimiento según la cantidad de hilos, mantenibilidad, complejidad y correctitud de ambos resultados, así como una comparativa de costos de los resultados de cada modelo de IA.

III. MÉTODOS

a. Flujo de desarrollo de la IA

El desarrollo de la aplicación se realiza por medio del Workflow sugerido por Anthropic “Explore-Plan-Execute-Commit” [1], donde se utiliza un modelo de mayor capacidad para las etapas de exploración y planeación que genera un plan resultante para que, en otra sesión con contexto limpio, el mismo modelo o uno de menor capacidad, se encargue de la ejecución.

Siguiendo este flujo, para la primera etapa utilizamos el modelo de Anthropic Claude Opus 4.7 [2], modo thinking con effort high, el modelo más avanzado hasta la fecha de la empresa, configurado con un alto nivel de razonamiento, para que, a partir de un prompt técnico base, escrito por un humano, genere un prompt técnico detallado de mayor exactitud. Este prompt es utilizado en la etapa de ejecución como entrada de ambos modelos, el mencionado previamente, y el modelo Claude Haiku 4.5 [2], el modelo más barato de la empresa, para generar ambos resultados de la aplicación.

La herramienta de Vibe Coding utilizada para esto es Claude Code [3], de la empresa Anthropic, que permite ejecutar agentes de IA que ofrecen un mejor rendimiento en la entrega de software.

b. Prompt Técnico

En línea con lo mencionado, se utilizan dos prompts, el inicial para la Planeación y el refinado para la Ejecución

PROMPT BASE PARA ETAPA DE PLANEACIÓN

Lenguaje de programación

Java

Plataforma

El programa debe ejecutarse en Windows

Funciones a implementar

Desarrollar un programa local de prueba que simule ser un servidor que recibe posts de usuarios de Twitter, y realice un análisis de cada uno, incluyendo lo siguiente:

- *Análisis de sentimiento, para clasificar si un tweet es positivo negativo o neutro.*
- *Clasificación de contenido prohibido, comparando contra una lista de palabras ya definida, devolviendo diferentes categorías: violencia, odio, adulto, etc.*
- *Detección de posible spam, de forma simple, usando regex para detectar palabras claves, ratio mayúsculas/minúsculas, longitud anómala. Devuelve un score de 0 a 1.*
- *Finalmente, debe determinar si el post está aprobado o rechazado.*
- *Guardar los resultados finales en un array, en un archivo de resultados en formato json*

Requisitos específicos

- *El programa debe utilizar concurrencia, con un set de thread pools para procesar los diferentes posts*

Restricciones

- *El programa debe ser simple, a fines demostrativos, que permita enfocarnos en el análisis del software.*

A partir de este prompt, el modelo Opus 4.7 con Plan Mode genera el prompt técnico refinado utilizado en la etapa de ejecución, el cual será adjunto debido a su extensión.

d.Métricas de Software y Uso

- Complejidad Ciclomática: Se aplica la Complejidad Ciclomática de McCabe extendida [4], que cuenta tanto las estructuras de control (if, for, while, catch) como los operadores lógicos compuestos (&&, ||) dentro de expresiones booleanas, ya que cada operador introduce un camino de ejecución independiente que el anterior omite. Este método es el estándar en la industria porque su valor correlaciona directamente con el esfuerzo de un nuevo desarrollador para comprender el código, la cantidad mínima de casos de prueba para cubrir el código y la probabilidad de defectos (estudios empíricos de Basili & Perricone muestran correlación positiva para $CC > 10$) [5].

Para realizar este cálculo de forma automática, se utiliza la herramienta PMD [6], y se consideran los resultados conforme a la siguiente clasificación [7]:

CC	Nivel de Complejidad
1-5	Bajo — testeabilidad alta
6-10	Moderado — tolerable

11–20	Alto — considerar refactoring
> 20	Muy alto — crítico

- Rendimiento del programa y uso de threads: se aplica análisis estático del código fuente complementado con profiling en runtime con librerías de medición de Java, para poder calcular el consumo de CPU, memoria, y la cantidad de threads, evaluando así la performance del programa.

- Correctitud de la concurrencia: se utiliza la herramienta SpotBugs [8], que posee detectores especializados de patrones incorrectos de concurrencia, como detección de Deadlocks, acceso sin sincronización, problemas de Locks, etc. Esto permite analizar la calidad de la concurrencia implementada por la IA.

- Consumo de IA: se calcula el consumo de tokens, el gasto estimado en dólares y el tiempo de ejecución de la IA, por medio del comando `/usage` que provee Claude Code, para así evaluar el costo asociado al uso de estas herramientas. A su vez, se contará la cantidad de iteraciones requeridas por el modelo para alcanzar el resultado.

IV. RESULTADOS Y OBJETIVOS

A continuación, se realiza una comparación entre los resultados ofrecidos por ambos modelos conforme a las métricas anteriores. Para ello, se utiliza un archivo de entrada *posts* idéntico para ambos programas, esperando obtener un resultado similar.

a. Resultados de Opus 4.7

Complejidad Ciclomática

- CC mínima: 1
- CC máxima: 12
- CC promedio: 4.7

El 93% de los métodos cae en rango bajo-moderado. `calcularSpamScore()` (CC=12) es el único que roza el umbral alto y candidato a refactoring.

Rendimiento del programa y uso de threads

Cantidad de posts procesados: 100

Cantidad de threads lanzados por la aplicación: 31 – cantidad definida de forma estática, no se lanzan dinámicamente

Tiempo total de la aplicación: 0.369 s

Memoria heap reservada por la aplicación: 238.00 MB

Correctitud de Concurrencia

En base al análisis de SpotBugs, junto a una comprobación estática, se puede observar que la concurrencia ha sido aplicada de forma adecuada, utilizando una arquitectura de pipeline con worker threads, que no presenta deadlock debido a que no se utilizan estructuras bloqueantes ni esperas circulares debido a la estructura del pipeline, tampoco se identifican problemas de race conditions o de sincronización debido al uso de estructuras thread-safe, o un diseño del programa que garantiza el acceso seguro a los posibles recursos compartidos, como los posts.

Consumo de IA

El costo de este programa se divide en la etapa de Planificación (transversal al resultado de ambos modelos) y en la etapa de Ejecución.

Consumo del Planificador:

- Coste: \$2.38
- Uso de Tokens: 39800
- Tiempo de respuesta: 8m 2s
- Iteraciones: 1

Consumo del Ejecutor:

- Coste: \$4.88
- Uso de Tokens: 94700
- Tiempo de respuesta: 17m 9s
- Iteraciones: 1

Consumo Total:

- Coste: \$7.26
- Uso de Tokens: 134500
- Tiempo de respuesta: 25m 11s
- Iteraciones: 1

b.Resultados de Haiku 4.5

Complejidad Ciclomática

- CC mínima: 1
- CC máxima: 10
- CC promedio: 2.12

El 90.2% de los métodos cae en rango bajo-moderado. calcularSpamScore() (CC=10) y Main() /(CC=9) son los métodos que se aproximan al umbral alto.

Rendimiento del programa y uso de threads

Cantidad de posts procesados: 100

Cantidad de threads lanzados por la aplicación: 31 – cantidad definida de forma estática, no se lanzan dinámicamente

Tiempo total de la aplicación: 0.254 s

Memoria heap reservada por la aplicación: 238.00 MB

Correctitud de Concurrencia

Al igual que en el caso anterior, el análisis de SpotBugs detectó que no hay problemas de Deadlocks o Race Condition, debido al uso adecuado de la concurrencia y de la implementación de estructuras preparadas para la misma.

Consumo de IA

El costo de este programa se divide en la etapa de Planificación (transversal al resultado de ambos modelos) y en la etapa de Ejecución.

Consumo del Planificador:

- Coste: \$2.38
- Uso de Tokens: 39800
- Tiempo de respuesta: 8m 2s
- Iteraciones: 1

Consumo del Ejecutor:

- Coste: \$0.50
- Uso de Tokens: 27800
- Tiempo de respuesta: 3m 49s
- Iteraciones: 1

Consumo Total:

- Coste: \$2.88
- Uso de Tokens: 67600
- Tiempo de respuesta: 11 51s
- Iteraciones: 1

V. CONCLUSIONES

Como se ha podido observar, ambos resultados han sido satisfactorios, ofreciendo una buena calidad de código y una sólida implementación de concurrencia, utilizando patrones y estructuras de datos adecuadas para la programación concurrente.

Cabe destacar los resultados provistos por el modelo Haiku, que ha dado una complejidad ligeramente inferior al modelo Opus y una mayor velocidad, logrando estos resultados con un costo total de \$2.88, frente a los \$7.26 consumidos por el modelo más potente.

Podemos concluir que el uso del workflow Explore-Plan-Execute-Commit con un modelo más poderoso para la planeación, y un modelo más eficiente para la ejecución, junto a un prompt adecuado y bien estructurado, ofrece un resultado de gran valor y eficiente en el uso de recursos, un punto a considerar en tiempos donde el precio de la IA tiende a incrementar, y con ello, la cautela a la hora de escoger las herramientas a utilizar en un desarrollo.

VI. REFERENCIAS Y BIBLIOGRAFÍA

A. Referencias bibliográficas:

- [1] Anthropic, "Claude Code Best Practices: Explore, plan, code, commit,". Disponible en: <https://code.claude.com/docs/en/best-practices#explore-first-then-plan-then-code>. [Accedido: 25-may-2026].
- [2] Anthropic, "Models overview," Claude Developer Platform. Disponible en: <https://platform.claude.com/docs/en/about-claude/models/overview>. [Accedido: 25-may-2026].
- [3] Anthropic, "Claude Code,". Disponible en: <https://www.anthropic.com/product/claude-code>. [Accedido: 25-may-2026].
- [4] T. J. McCabe, "A Complexity Measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308-320, dic. 1976.
- [5] V. R. Basili y B. T. Perricone, "Software errors and complexity: an empirical investigation," *Communications of the ACM*, vol. 27, no. 1, pp. 42-52, ene. 1984. Disponible en: <https://www.cs.umd.edu/~basili/publications/journals/J20.pdf>. [Accedido: 25-may-2026].
- [6] PMD, "PMD Source Code Analyzer," GitHub. Disponible en: <https://github.com/pmd/pmd>. [Accedido: 25-may-2026].
- [7] IBM, "Cyclomatic complexity," *IBM Rational Asset Analyzer 6.1.0 Documentation*. Disponible en: <https://www.ibm.com/docs/en/raa/6.1.0?topic=metrics-cyclomatic-complexity>. [Accedido: 25-may-2026].
- [8] SpotBugs, "SpotBugs - Find bugs in Java programs,". Disponible en: <https://spotbugs.github.io/>. [Accedido: 25-may-2026].

B. Bibliografía:

- [B1] B. Goetz, T. Peierls, J. Bloch, J. Bowbeer, D. Holmes y D. Lea, *Java Concurrency in Practice*, 1ª ed. Boston, MA: Addison-Wesley Professional, 2006.
- [B2] D. Lea, *Concurrent Programming in Java: Design Principles and Patterns*, 2ª ed. Boston, MA: Addison-Wesley Professional, 1999

